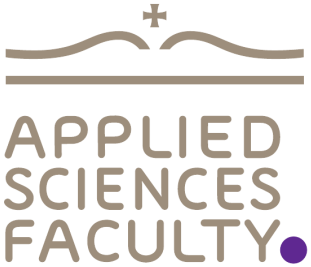
C++ — додаток. Binutils та GCC, GDB, FASM





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

- GNU Binary Utilities – binutils, це набір інструментів для маніпуляції двійковим кодом – виконавчими файлами, об'єктними файлами, динамічними та статичними бібліотеками тощо.
- Поміж них також as – GNU Assembler.
- Офіційна адреса: `https://www.gnu.org/software/binutils/`
- Там, де це можливо, вони кросплатформові.
- Можливості частини із них, наприклад, objdump і readelf[1], перетинаються.
- Деякі із них розглянуто нижче. Архіватор статичних бібліотек, ar, описано у презентації про бібліотеки. Там же є приклади ld – лінкера.

[1] Якщо коротко, objdump вміє дизасемблювати, а readelf ні, але не потребує `https://en.wikipedia.org/wiki/Binary_File_Descriptor_library`.

nm – показати символи із об'єктного файлу

- Для об'єктного файлу, згенерованого для коду C:

```
$ nm file1.o
0000000000000000 T function_from_file1
                 U _GLOBAL_OFFSET_TABLE_
0000000000000000 r .LC0
                 U __printf_chk
```

T – символ визначено у цьому файлі, U – символ НЕ визначено.

- Для C++ буде трохи гірше:

```
$ nm file1.o
                 U __cxa_atexit
                 U __dso_handle
                 U _GLOBAL_OFFSET_TABLE_
0000000000000000 t _GLOBAL__sub_I__Z19function_from_file11X
0000000000000000 r .LC0
0000000000000000 T _Z19function_from_file11X
                 U _ZNKSt5ctypeIcE13_M_widen_initEv
0000000000000000 W _ZNKSt5ctypeIcE8do_widenEc
                 U _ZNSo3putEc
                 U _ZNSo5flushEv
                 U _ZNSt8ios_base4InitC1Ev
                 U _ZNSt8ios_base4InitD1Ev
```

- Допоможе demangling:

```
$ nm file1.o -C
                 U __cxa_atexit
                 U __dso_handle
                 U _GLOBAL_OFFSET_TABLE_
0000000000000000 t _GLOBAL__sub_I__Z19function_from_file11X
0000000000000000 r .LC0
0000000000000000 T function_from_file1(X)
                 U std::ctype<char>::_M_widen_init() const
0000000000000000 W std::ctype<char>::do_widen(char) const
                 U std::ostream::put(char)
                 U std::ostream::flush()
                 U std::ios_base::Init::Init()
                 U std::ios_base::Init::~Init()
                 U std::basic_ostream<char, std::char_traits<char>
                 U std::__throw_bad_cast()
                 U std::cout
0000000000000000 b std::__ioinit
```

Формати для nm

- В першу чергу – об'єктні файли, ".o".
- Розуміє їх архіви – статичні бібліотеки, ".a".
- Динамічні бібліотеки відповідних платформ: .dll, .so.
- Виконавчі файли відповідних платформ (PE, ELF тощо).
- Через особливість реалізації, в PE-файлах невизначених символів не буде. ELF посилатиметься на символи із динамічних бібліотек як на не визначені.

Види символів nm

- Велика літера опису символу означає, що він – глобальний (external), мала – локальний (static).
- A – символ має фіксовану адресу, лінкер його не змінюватиме. C – common-символ. (Див. Linkers and Loaders за подробицями).
- B/b – символ із BSS. D/d – із секції даних, R/r – із секції даних тільки для читання. На деяких системах дані можуть помічатися і по іншому (G/g, S/s – малі дані, I – посилання на інший символ, скажімо).
- N – відладочний символ.
- T/t – символ у секції коду (text). T – експортується, t – локальний.
- U – символ не визначено.
- u – GNU extension, унікальний глобальний символ.
- V/v – слабкий (weak) символ, рівний 0, якщо не визначено. W/w – слабкий символ, якщо не визначено, визначається системо-залежним способом.
- Існують інші, більш спеціалізовані, прапорці. Скажімо, для Windows PE, "i" означає – символ, потрібний для реалізації DLL.

Опції nm

- "-A" – виводити перед кожним символом ім'я його файлу.
- "-a" – показувати відладочні символи.
- "-C" – вже згадувана demangle, розплутування імен C++.
- "-D" – показувати лише експортовані динамічні символи, має сенс лише для динамічних бібліотек.
- "-g" – показувати лише зовнішні (external) символи.
- "-l" – для файлів із відладочною інформацією, намагатися знайти файл та рядок у ньому, де визначено цей символ.
- "-n" – сортувати за адресою, а не за іменем. "-p" – взагалі не сортувати. "-r" – зворотній порядок сортування.
- "-u" – виводити лише не визначені символи.
- "–defined-only" – виводили лише визначені.
- Більшість опцій мають синоніми, тут їх не наводжу.
- За рештою опції див. документацію або "man nm".

ldd

- ldd виводить динамічні бібліотеки, потрібні даному виконавчому файлу чи динамічній бібліотеці:

```
$ ldd bin/hellomake.exe
        ntdll.dll => /c/WINDOWS/SYSTEM32/ntdll.dll (0x7ffb54e20000)
        KERNEL32.DLL => /c/WINDOWS/System32/KERNEL32.DLL (0x7ffb53f20000)
        KERNELBASE.dll => /c/WINDOWS/System32/KERNELBASE.dll (0x7ffb51240000)
        msvcrt.dll => /c/WINDOWS/System32/msvcrt.dll (0x7ffb54d30000)
        libstdc++-6.dll => /mingw64/bin/libstdc++-6.dll (0x6fc40000)
        USER32.dll => /c/WINDOWS/System32/USER32.dll (0x7ffb52860000)
        win32u.dll => /c/WINDOWS/System32/win32u.dll (0x7ffb51540000)
        GDI32.dll => /c/WINDOWS/System32/GDI32.dll (0x7ffb52780000)
        gdi32full.dll => /c/WINDOWS/System32/gdi32full.dll (0x7ffb52110000)
        msvcp_win.dll => /c/WINDOWS/System32/msvcp_win.dll (0x7ffb515c0000)
        ucrtbase.dll => /c/WINDOWS/System32/ucrtbase.dll (0x7ffb51710000)
        libwinpthread-1.dll => /mingw64/bin/libwinpthread-1.dll (0x64940000)
        libgcc_s_seh-1.dll => /mingw64/bin/libgcc_s_seh-1.dll (0x61440000)

$ ldd hellomake
        linux-vdso.so.1 (0x00007ffff8c42000)
        libstdc++.so.6 => /usr/lib/x86_64-linux-gnu/libstdc++.so.6 (0x00007f85c0c8
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f85c0880000)
        libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6 (0x00007f85c04e0000)
        /lib64/ld-linux-x86-64.so.2 (0x00007f85c1400000)
        libgcc_s.so.1 => /lib/x86_64-linux-gnu/libgcc_s.so.1 (0x00007f85c02c0000)
```

ldd

- Ремарка: ldd покладається на динамічний завантажувач[2].
- Опції:
  - ◇ "-v" – виводити детальну інформацію.
  - "-u" – виводити інформацію про не використані залежності.
  - "-r", "-d" – для ELF, виконати повну релокацію чи тільки для даних та повідомити про відсутні символи.
  - `LD_DEBUG=libs ldd <executable>` – спосіб подивитися на подробиці процесу пошуку.
- Ремарка 2: ldd не зважає на suid/guid прапорці, і робить вигляд, що $ORIGIN теж використовуватиметься для пошуку.
- Суміжна утиліта, pldd, дозволяє подивитися бібліотеки, завантажені процесом, що виконується.
- "Dependency Walker" – зручна GUI-утиліта під Windows. На жаль, останнім часом (2020/Windows 10) трохи капризує на kernel32.dll.

[2] Детальніше див. `https://man7.org/linux/man-pages/man1/ldd.1.html`.

size

- size виводить розмір секцій у виконавчому, об'єктному файлі чи бібліотеці:

```
<...>/make_demo$ size libfiles.so
   text    data     bss     dec     hex filename
   2484     616       8    3108     c24 libfiles.so
<...>/make_demo$ size libfiles.a
   text    data     bss     dec     hex filename
    384       8      16     408     198 file1.o (ex libfiles.a)
    400       8      16     424     1a8 file2.o (ex libfiles.a)
<...>/make_demo$ size main
   text    data     bss     dec     hex filename
   2882     704     280    3866     f1a main
```

- https://sourceware.org/binutils/docs-2.34/binutils/size.html.

c++filt

- c++filt здійснює demangling для імен C++ та Java. Може приймати імена з командного рядка чи з консолі.

```
<...>/make_demo$ c++filt _Z19function_from_file11X
function_from_file1(X)
<...>/make_demo$ echo '_ZStlsISt11char_traitsIcEERSt13basic_ostrea
                          | c++filt
std::basic_ostream<char, std::char_traits<char> >&
 std::operator<< <std::char_traits<char> >(
  std::basic_ostream<char, std::char_traits<char> >&,
    char const*)
```

- https://sourceware.org/binutils/docs-2.34/binutils/c_002b_002bfilt.html.

objdump

- objump дозволяє вивести практично будь-яку інформацію про об'єктний файл – від його типу до дизасемблювання вмісту.
- Активно використовується в презентації x86_arch.pdf. Також, виконайте "objdump –help".

Заголовки файлів – ключ -f

```
<...>/make_demo$ objdump -f main
main:     file format elf64-x86-64
architecture: i386:x86-64, flags 0x00000150:
HAS_SYMS, DYNAMIC, D_PAGED
start address 0x0000000000001120

<...>/make_demo$ objdump -f main.exe
main.exe:     file format pei-x86-64
architecture: i386:x86-64, flags 0x0000013a:
EXEC_P, HAS_DEBUG, HAS_SYMS, HAS_LOCALS, D_PAGED
start address 0x00000000004014e0

<...>/make_demo$ objdump -f main_file.o
main_file.o:     file format elf64-x86-64
architecture: i386:x86-64, flags 0x00000011:
HAS_RELOC, HAS_SYMS
start address 0x0000000000000000

<...>/make_demo$ objdump -f libfiles.so
libfiles.so:     file format elf64-x86-64
architecture: i386:x86-64, flags 0x00000150:
HAS_SYMS, DYNAMIC, D_PAGED
start address 0x00000000000010c0
```

Вміст заголовків файлів – ключ -p

```
<...>/make_demo$ objdump -p main

main:     file format elf64-x86-64

Program Header:
    PHDR off    0x0000000000000040 vaddr 0x0000000000000040 paddr 0x0000000000000040 align 2**3
         filesz 0x00000000000002d8 memsz 0x00000000000002d8 flags r--
  INTERP off    0x0000000000000318 vaddr 0x0000000000000318 paddr 0x0000000000000318 align 2**0
         filesz 0x000000000000001c memsz 0x000000000000001c flags r--
    LOAD off    0x0000000000000000 vaddr 0x0000000000000000 paddr 0x0000000000000000 align 2**12
         filesz 0x0000000000000940 memsz 0x0000000000000940 flags r--
.................skipped.....................................

Dynamic Section:
  NEEDED               libfiles.so
  NEEDED               libstdc++.so.6
  NEEDED               libc.so.6
  INIT                 0x0000000000001000
  FINI                 0x0000000000001368
  INIT_ARRAY           0x0000000000003d50
  INIT_ARRAYSZ         0x0000000000000010
.................skipped.....................................

Version References:
  required from libstdc++.so.6:
    0x08922974 0x00 03 GLIBCXX_3.4
  required from libc.so.6:
    0x0d696914 0x00 04 GLIBC_2.4
    0x09691a75 0x00 02 GLIBC_2.2.5
```

Інформація про секції файлу – ключ -h

```
<...>/make_demo$ objdump -h main

main:     file format elf64-x86-64

Sections:
Idx Name          Size      VMA               LMA               File off  Algn
  0 .interp       0000001c  0000000000000318  0000000000000318  00000318  2**0
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  1 .note.gnu.property 00000020  0000000000000338  0000000000000338  00000338  2**3
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  2 .note.gnu.build-id 00000024  0000000000000358  0000000000000358  00000358  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  3 .note.ABI-tag 00000020  000000000000037c  000000000000037c  0000037c  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  4 .gnu.hash     00000028  00000000000003a0  00000000000003a0  000003a0  2**3
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  5 .dynsym       00000180  00000000000003c8  00000000000003c8  000003c8  2**3
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  6 .dynstr       000001bd  0000000000000548  0000000000000548  00000548  2**0
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  7 .gnu.version  00000020  0000000000000706  0000000000000706  00000706  2**1
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  8 .gnu.version_r 00000050  0000000000000728  0000000000000728  00000728  2**3
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  9 .rela.dyn     00000120  0000000000000778  0000000000000778  00000778  2**3
```

Інформація про всі заголовки файлу – ключ -x

```
<...>/make_demo$ objdump -x main
main:     file format elf64-x86-64
.................skipped.....................................

Program Header:
    PHDR off    0x0000000000000040 vaddr 0x0000000000000040 paddr 0x0000000000000040 align 2**3
         filesz 0x00000000000002d8 memsz 0x00000000000002d8 flags r--
.................skipped.....................................

Dynamic Section:
  NEEDED               libfiles.so
  NEEDED               libstdc++.so.6
  NEEDED               libc.so.6
  INIT                 0x0000000000001000
.................skipped.....................................

Version References:
  required from libstdc++.so.6:
.................skipped.....................................

Sections:
Idx Name          Size      VMA               LMA               File off  Algn
  0 .interp       0000001c  0000000000000318  0000000000000318  00000318  2**0
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
.................skipped.....................................

SYMBOL TABLE:
0000000000000318 l    d  .interp        0000000000000000              .interp
0000000000000338 l    d  .note.gnu.property     0000000000000000              .note.gnu.property
0000000000000358 l    d  .note.gnu.build-id     0000000000000000              .note.gnu.build
```

Дизасемблювання, AT&T стиль – ключ -d

```
<...>/make_demo$ objdump -d main
main:     file format elf64-x86-64
Disassembly of section .init:

0000000000001000 <_init>:
    1000:       f3 0f 1e fa             endbr64
    1004:       48 83 ec 08             sub    $0x8,%rsp
    1008:       48 8b 05 d9 2f 00 00    mov    0x2fd9(%rip),%rax        # 3fe8 <__gmon_start__>
..................skipped.........................................

Disassembly of section .plt.sec:
00000000000010b0 <_Z19function_from_file11X@plt>:
    10b0:       f3 0f 1e fa             endbr64
    10b4:       f2 ff 25 d5 2e 00 00    bnd jmpq *0x2ed5(%rip)        # 3f90 <_Z19function_from_fi
    10bb:       0f 1f 44 00 00          nopl   0x0(%rax,%rax,1)

..................skipped.........................................

Disassembly of section .text:
0000000000001120 <_start>:
    1120:       f3 0f 1e fa             endbr64
    1124:       31 ed                   xor    %ebp,%ebp
    1126:       49 89 d1                mov    %rdx,%r9
    1129:       5e                      pop    %rsi
    112a:       48 89 e2                mov    %rsp,%rdx
    112d:       48 83 e4 f0             and    $0xfffffffffffffff0,%rsp
    1131:       50                      push   %rax
    1132:       54                      push   %rsp
    1133:       4c 8d 05 26 02 00 00    lea    0x226(%rip),%r8        # 1360 <__libc_csu_fini>
```

Дизасемблювання, Intel стиль – ключі -M intel -d

```
<...>/make_demo$ objdump  -M intel -d main
main:     file format elf64-x86-64
Disassembly of section .init:

0000000000001000 <_init>:
    1000:       f3 0f 1e fa             endbr64
    1004:       48 83 ec 08             sub    rsp,0x8
    1008:       48 8b 05 d9 2f 00 00    mov    rax,QWORD PTR [rip+0x2fd9]        # 3fe8 <__gmon_st
..................skipped.........................................

Disassembly of section .plt.sec:
00000000000010b0 <_Z19function_from_file11X@plt>:
    10b0:       f3 0f 1e fa             endbr64
    10b4:       f2 ff 25 d5 2e 00 00    bnd jmp QWORD PTR [rip+0x2ed5]        # 3f90 <_Z19function_
    10bb:       0f 1f 44 00 00          nop    DWORD PTR [rax+rax*1+0x0]

..................skipped.........................................

Disassembly of section .text:
0000000000001120 <_start>:
    1120:       f3 0f 1e fa             endbr64
    1124:       31 ed                   xor    ebp,ebp
    1126:       49 89 d1                mov    r9,rdx
    1129:       5e                      pop    rsi
    112a:       48 89 e2                mov    rdx,rsp
    112d:       48 83 e4 f0             and    rsp,0xfffffffffffffff0
    1131:       50                      push   rax
    1132:       54                      push   rsp
    1133:       4c 8d 05 26 02 00 00    lea    r8,[rip+0x226]        # 1360 <__libc_csu_fini>
```

Дизасемблювання, виводячи відповідні рядки з джерельних текстів

Потребує файлу, скомпільованого із інформацією про відладку:

```
$ objdump -M intel -S library.o

library.o:     file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <library_function>:
#include "file1.h"

int global_var;

void library_function(int arg)
{
   0:   f3 0f 1e fa             endbr64
   4:   55                      push   rbp
   5:   48 89 e5                mov    rbp,rsp
   8:   48 83 ec 10             sub    rsp,0x10
   c:   89 7d fc                mov    DWORD PTR [rbp-0x4],edi
        printf("Argument: %i\n", arg);
   f:   8b 45 fc                mov    eax,DWORD PTR [rbp-0x4]
  12:   89 c6                   mov    esi,eax
  14:   48 8d 05 00 00 00 00    lea    rax,[rip+0x0]        # 1b <library_function+0x1b>
  1b:   48 89 c7                mov    rdi,rax
  1e:   b8 00 00 00 00          mov    eax,0x0
  23:   e8 00 00 00 00          call   28 <library_function+0x28>
        printf("Global var: %i\n", global_var);
  28:   8b 05 00 00 00 00       mov    eax,DWORD PTR [rip+0x0]        # 2e <library_function+0x2e>
  2e:   89 c6                   mov    esi,eax
```

Дизасемблювання, виводячи адресу кожного рядка

```
<...>/make_demo$ objdump -M intel -d --prefix-addresses main

main:     file format elf64-x86-64

Disassembly of section .init:
0000000000001000 <_init> endbr64
0000000000001004 <_init+0x4> sub    rsp,0x8
0000000000001008 <_init+0x8> mov    rax,QWORD PTR [rip+0x2fd9]        # 0000000000003fe8 <__gmon_s
000000000000100f <_init+0xf> test   rax,rax
0000000000001012 <_init+0x12> je     0000000000001016 <_init+0x16>
0000000000001014 <_init+0x14> call   rax
0000000000001016 <_init+0x16> add    rsp,0x8
000000000000101a <_init+0x1a> ret

Disassembly of section .plt:
0000000000001020 <.plt> push   QWORD PTR [rip+0x2f5a]        # 0000000000003f80 <_GLOBAL_OFFSET_TA
0000000000001026 <.plt+0x6> bnd jmp QWORD PTR [rip+0x2f5b]        # 0000000000003f88 <_GLOBAL_OFFS
.................skipped.....................................
```

Вивести вміст всіх секцій файлу – -s

```
<...>/make_demo$ objdump  -s main

main:     file format elf64-x86-64

Contents of section .interp:
 0318 2f6c6962 36342f6c 642d6c69 6e75782d  /lib64/ld-linux-
 0328 7838362d 36342e73 6f2e3200           x86-64.so.2.
Contents of section .note.gnu.property:
 0338 04000000 10000000 05000000 474e5500  ............GNU.
 0348 020000c0 04000000 03000000 00000000  ................
Contents of section .note.gnu.build-id:
 0358 04000000 14000000 03000000 474e5500  ............GNU.
 0368 aa408ca5 2b0e85a0 53c58398 4e52da50  .@..+...S...NR.P
 0378 29a25f0a                             ).._.
Contents of section .note.ABI-tag:
 037c 04000000 10000000 01000000 474e5500  ............GNU.
 038c 00000000 03000000 02000000 00000000  ................
Contents of section .gnu.hash:
 03a0 02000000 0e000000 01000000 06000000  ................
 03b0 00009100 01000000 0e000000 00000000  ................
 03c0 d065ce6d 15980c43                    .e.m...C
Contents of section .dynsym:
 03c8 00000000 00000000 00000000 00000000  ................
..................skipped.......................................
Contents of section .dynstr:
 0548 006c6962 66696c65 732e736f 005f5f67  .libfiles.so.__g
 0558 6d6f6e5f 73746172 745f5f00 5f49544d  mon_start__._ITM
 0568 5f646572 65676973 74657254 4d436c6f  _deregisterTMClo
 0578 6e655461 626c6500 5f49544d 5f726567  neTable._ITM_reg
```

Вивести таблицю динамічних символів – -T (див. також -t)

```
<...>/make_demo$ objdump  -T main

main:     file format elf64-x86-64

DYNAMIC SYMBOL TABLE:
0000000000000000      DF *UND*  0000000000000000              _Z19function_from_file11X
0000000000000000      DF *UND*  0000000000000000  GLIBCXX_3.4 _ZSt4endlIcSt11char_traitsIcEERSt13b
0000000000000000      DF *UND*  0000000000000000  GLIBC_2.2.5 __cxa_atexit
0000000000000000      DF *UND*  0000000000000000  GLIBCXX_3.4 _ZStlsISt11char_traitsIcEERSt13basic_
0000000000000000      DF *UND*  0000000000000000  GLIBCXX_3.4 _ZNSolsEPFRSoS_E
0000000000000000      DF *UND*  0000000000000000              _Z19function_from_file2v
0000000000000000      DF *UND*  0000000000000000  GLIBC_2.4   __stack_chk_fail
0000000000000000      DF *UND*  0000000000000000  GLIBCXX_3.4 _ZNSt8ios_base4InitC1Ev
0000000000000000  w   D  *UND*  0000000000000000              _ITM_deregisterTMCloneTable
0000000000000000      DF *UND*  0000000000000000  GLIBC_2.2.5 __libc_start_main
0000000000000000  w   D  *UND*  0000000000000000              __gmon_start__
0000000000000000  w   D  *UND*  0000000000000000              _ITM_registerTMCloneTable
0000000000000000      DF *UND*  0000000000000000  GLIBCXX_3.4 _ZNSt8ios_base4InitD1Ev
0000000000000000  w   DF *UND*  0000000000000000  GLIBC_2.2.5 __cxa_finalize
0000000000004040 g    DO .bss   0000000000000110  GLIBCXX_3.4 _ZSt4cout
```

Зверніть увагу: програма динамічно лінкується libfiles.so, у котрій є function_from_file1 і function_from_file2.

Вивести динамічні релокації – -R (всі – -r)

```
<...>/make_demo$ objdump  -R main

main:     file format elf64-x86-64

DYNAMIC RELOCATION RECORDS
OFFSET           TYPE              VALUE
0000000000003d50 R_X86_64_RELATIVE  *ABS*+0x0000000000001200
0000000000003d58 R_X86_64_RELATIVE  *ABS*+0x00000000000012ca
0000000000003d60 R_X86_64_RELATIVE  *ABS*+0x00000000000011c0
0000000000004008 R_X86_64_RELATIVE  *ABS*+0x0000000000004008
0000000000003fc8 R_X86_64_GLOB_DAT  __cxa_finalize@GLIBC_2.2.5
0000000000003fd0 R_X86_64_GLOB_DAT  _ZSt4endlIcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6_@GLI
0000000000003fd8 R_X86_64_GLOB_DAT  _ITM_deregisterTMCloneTable
0000000000003fe0 R_X86_64_GLOB_DAT  __libc_start_main@GLIBC_2.2.5
0000000000003fe8 R_X86_64_GLOB_DAT  __gmon_start__
0000000000003ff0 R_X86_64_GLOB_DAT  _ITM_registerTMCloneTable
0000000000003ff8 R_X86_64_GLOB_DAT  _ZNSt8ios_base4InitD1Ev@GLIBCXX_3.4
0000000000004040 R_X86_64_COPY     _ZSt4cout@@GLIBCXX_3.4
0000000000003f90 R_X86_64_JUMP_SLOT  _Z19function_from_file11X
0000000000003f98 R_X86_64_JUMP_SLOT  __cxa_atexit@GLIBC_2.2.5
0000000000003fa0 R_X86_64_JUMP_SLOT  _ZStlsISt11char_traitsIcEERSt13basic_ostreamIcT_ES5_PKc@GLIBC
0000000000003fa8 R_X86_64_JUMP_SLOT  _ZNSolsEPFRSoS_E@GLIBCXX_3.4
0000000000003fb0 R_X86_64_JUMP_SLOT  _Z19function_from_file2v
0000000000003fb8 R_X86_64_JUMP_SLOT  __stack_chk_fail@GLIBC_2.4
0000000000003fc0 R_X86_64_JUMP_SLOT  _ZNSt8ios_base4InitC1Ev@GLIBCXX_3.4
```

- Опція -D – вивести дизасемблювання всіх секцій. В нормі потрібно рідко – не всі секції містять код, а ті, що містять, дизасемблюватиме і -d.
- -g – вивести інформацію для відлагодження.
- -t – вивести таблицю символів.
- Уточнити секцію, яка цікавить – опція -j: `objdump -d -j.plt main`.
- Детальніше: `https://sourceware.org/binutils/docs-2.34/binutils/objdump.html`.

readelf

- Можливості readelf та objdump перетинаються, тому лише приклад:

```
<...>/make_demo$ readelf -h main
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              DYN (Shared object file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x1120
  Start of program headers:          64 (bytes into file)
  Start of section headers:          36632 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:         13
  Size of section headers:           64 (bytes)
  Number of section headers:         36
  Section header string table index: 35
```

- Детальніше: https://sourceware.org/binutils/docs-2.34/binutils/readelf.html.

Bloaty

- Bloaty дозволяє побачити, на що саме йде місце у виконавчому файлі.
- Приклад з "Honey, I shrunk {fmt}: bringing binary size to 14k and ditching the C++ runtime":

```
$ bloaty -d symbols a.out

    FILE SIZE        VM SIZE
 --------------  --------------
  43.8%  41.1Ki  43.6%  29.0Ki    [121 Others]
   6.4%  6.04Ki   8.1%  5.42Ki    fmt::v11::detail::do_write_float<>()
   5.9%  5.50Ki   7.5%  4.98Ki    fmt::v11::detail::write_int_noinline<>()
   5.7%  5.32Ki   5.8%  3.88Ki    fmt::v11::detail::write<>()
   5.4%  5.02Ki   7.2%  4.81Ki    fmt::v11::detail::parse_replacement_field<>()
   3.9%  3.69Ki   3.7%  2.49Ki    fmt::v11::detail::format_uint<>()
   3.2%  3.00Ki   0.0%       0    [section .symtab]
   2.7%  2.50Ki   0.0%       0    [section .strtab]
   2.3%  2.12Ki   2.9%  1.93Ki    fmt::v11::detail::dragonbox::to_decimal<>()
```

- Детальніше: `https://github.com/google/bloaty`.

Інші

- **strip** видаляє із бінарного, зокрема – виконавчого, файлу все не потрібне для його виконання – відладочну інформацію, інформацію про символи тощо. Зменшує розмір файлів та ускладнює reverse engineering. **Увага** – заміняє вихідний файл. `https://sourceware.org/binutils/docs-2.34/binutils/strip.html#st`.
- strings виводить всі стрічки в бінарному файлі. Що для нього стрічка – див. `https://sourceware.org/binutils/docs-2.34/binutils/strings.html`. Можна попросити про це ж лінкер, опцією ”-s“.
- addr2line перетворює адресу у номер рядка. Адресу можна отримати із дизасемблу, отриманого від objdump, а далі перетворити її в рядок вихідного файлу (C/C++/...) addr2line. Потребує інформацію про відладку. Приклад: `https://www.geeksforgeeks.org/addr2line-command-in-linux-with-examples/`, документація: `https://sourceware.org/binutils/docs-2.34/binutils/addr2line.html`.

Інші

- objcopy – копіювання об'єктних файлів із внесенням до них змін. `https://sourceware.org/binutils/docs-2.34/binutils/objcopy.html`.
- windmc і windres – робота із ресурсами MS Windows[3].
- Сторонній пакет, checksec – корисний інструмент, скрип, який агрегує вивід різних утиліт типу readelf. Подробиці: `https://habr.com/ru/company/macloud/blog/562420/`.
- Список – не вичерпний. Більше тут: `https://sourceware.org/binutils/docs-2.34/binutils/index.html`.
- `https://github.com/noseglasses/elf_diff` – утиліта для аналізу різниці між двома (бажано – спорідненими та схожими) elf-файлами. Часто – дуже корисна річ!
- `https://lief.re/` – "LIEF : Library to Instrument Executable Formats", дуже потужна бібліотека.
- `https://www.sweetscape.com/010editor/` – текстовий редактор з потужним HEX-редактором, який включає сотні готових бінарних шаблонів і можливість писати свої: `https://www.sweetscape.com/010editor/repository/templates/`. ~~На жаль, платний.~~

[3] `https://en.wikipedia.org/wiki/Resource_(Windows)`.

Посилання

- "ELF's ways to combine potentially non-unique objects"
  - "IA-64 gABI Issue 72: COMDAT"
  - "COMDAT and section group "

Загальні опції GCC

- -c – лише компілювати/асемблювати, не виконувати лінкування. "-S" – і не асемблювати, "-E" – обмежитися препроцесингом.
- -o – вказати ім'я вихідного файлу.
- -x – вказати мову (по замовчуванню, орієнтується на розширення). Мови: c, c++, assembler, ada, d, f77, f95, go та ряд специфічних, типу c++-cpp-output чи assembler-with-cpp.
- -v – виводити багато подробиць під час компіляції.
- –version – версія компілятора.
- –help – підказка.
- -Wall – ввімкнути всі попередження. "-Wextra" – деякі додаткові попередження. Окремі попередження можна вмикати відповідними опціями: "-Wunused-value", а вимикати, додавши "no-": "-Wno-unused-value". " -Werror" – вважати всі попередження помилками, "-Werror=<warning>" – зробити помилкою конкретне попередження, наприклад "-Werror=switch".

Генерація коду

- -fPIC – генерувати PIC-код (Position Independent Code).
- -g – додавати інформацію для відладки. Слід передавати і під час компіляції і під час лінкування.
- -pg – інструментувати код для профілювання (див. gprof). Слід передавати і під час компіляції і під час лінкування.
- -march=xxx – вказати архітектуру та, за потреби, уточнення до неї: native (код для того ж процесора, на якому компілюємо), x86-64, i386, i486, ..., sandybridge, skylake, znver1, і т.д. і т.п. (Див. `https://gcc.gnu.org/onlinedocs/gcc/x86-Options.html`, `https://gcc.gnu.org/onlinedocs/gcc/ARM-Options.html`, `https://gcc.gnu.org/onlinedocs/gcc/AVR-Options.html` і т.д.). Код може не виконуватися на інших процесорах. "-mtune" оптимізує код під відповідну архітектуру (але, без -march не використовуватиме спеціальних інструкцій), приймає те ж, що і -march, а також – generic чи intel.
- Для x86: Здоровенний список, що вказує відповідні дозволені розширення: -msse4, -msse4.2, -mavx, -msha і т.д. і т.п., див. посилання вище. Керування FPU: -mfpmath=, де значення – 387, sse, both (має синоніми типу sse+387). Для ARM: -mfloat-abi=, де значення soft, softfp чи hard, а -march= відображає рівень архітектури, типу armv7 чи armv7e-m.

Пошук різноманітних частин проекту

- -I – шлях до файлів заголовків (include path). Може вказуватися декілька раз. Також, після вказаних цією опцією шляхів, GCC дивитиметься на змінні середовища `CPATH` та, в залежності від мови, `C_INCLUDE_PATH` чи `CPLUS_INCLUDE_PATH`. В цих змінних шляхи слід відділяти двокрапкою.
- -L – шлях пошуку бібліотек. Як і для -I, може бути декілька, після них ще загляне у змінну `LIBRARY_PATH`.
- -l – вказати бібліотеку для лінкування. Якщо бібліотека називається libname.a, слід вказати лише -lname, "lib" і ".a' чи ".so.a" будуть додані автоматично.
- **Увага!** Лінкер часто проглядає бібліотеки виключно зліва направо, тому порядок їх вказання опцією -l важливий! Якщо бібліотека A залежить від B, то слід спершу вказати A, а потім B. У складних випадках (скажімо, циклічних залежностей), ту ж бібліотеку доводиться вказувати декілька разів.
- PATH використовується для пошуку виконавчих файлів та динамічних бібліотек самого компілятора.

- Пам'ятайте, що змінні слід експортувати, щоб GCC їх побачив, або встановлювати безпосередньо перед викликом:

```
C_INCLUDE_PATH=$C_INCLUDE_PATH:/my/path/include
export C_INCLUDE_PATH
gcc <options>
або
C_INCLUDE_PATH=$C_INCLUDE_PATH:/my/path/include gcc <options>
```

- Шляхи, за якими компілятор шукатиме, можна побачити так:

```
gcc --print-search-dirs
```

- Визначені макроси:

```
$ touch empty.c
$ gcc -v -dD -E empty.c
```

Оптимізація

- -O0 – без оптимізації.
- -O1, -O2, -O3 – різні рівні оптимізації: базові оптимізації; всі оптимізації, що не збільшують розмір; ряд додаткових дорогих[4] оптимізацій, відповідно.
- -Os – оптимізувати, надаючи перевагу меншому розміру коду.
- -Ofast – максимально швидкий код, однак, із порушенням певних стандартів. Зокрема, включає -ffast-math.
- -Og – оптимізація, що не заважає відладці.
- -fexpensive-optimizations – додаткові, дорогі, оптимізації.
- Кожна із опції вище включає багато різних оптимізацій. Можна також вибирати їх детальніше, наприклад: -fomit-frame-pointer/-fno-omit-frame-pointer.
- -flto – "Link Time Optimization", глобальна оптимізація цілої програми. Див. `https://gcc.gnu.org/wiki/LinkTimeOptimization`.
- Більшість цих опції слід передавати і під час лінкування!

[4] З точки зору часу компіляції та потреб у RAM.

-ffast-math

- -ffast-math – прискорює floating-point арифметику, ціною відхилення від стандартів, зокрема IEEE 754. Зокрема:
- Дозволяє перевпорядковувати FP-операції, що може трохи змінювати результат.
- Не змінює errno після більшості FP-команд – це економить час на звертання до thread-local змінної.
- Вважає всі обчислення скінченими – що ніколи не трапляються NaN чи Inf.
- Виконує деякі оптимізації обчислення ділення та квадратного кореня (через обчислення оберненого), що можуть зменшувати точність.
- Ігнорує можливість апаратних переривань (у відповідь на NaN чи ділення на нуль, скажімо).
- І т.д.
- Зазвичай, результат залишається правильним, а код стає помітно швидшим. Однак, якщо код покладається на IEEE 754, зокрема, на суворе слідування правилам заокруглення, потребує errno і т.д. – можуть початися проблеми. Див. також `https://gcc.gnu.org/wiki/FloatingPointMath`.

Санітайзери

- Нові версії GCC підтримують багато "санітайзерів" – спеціальних засобів інструментування коду, що допомагають виявити типові помилки. Див. `https://github.com/google/sanitizers/wiki`.
- -fsanitize=address – вставляє код детектування помилок роботи із пам'яттю, зокрема – вихід за межі масивів та використання після звільнення. Роботою можна керувати за допомогою змінної середовища `ASAN_OPTIONS`. Подробиці див. `https://github.com/google/sanitizers/wiki/AddressSanitizer`.
- -fsanitize=thread – детектування проблем із багатопоточністю. Див. `https://github.com/google/sanitizers/wiki#threadsanitizer`.
- -fsanitize=leak – детектування витікання пам'яті, якщо не використано -fsanitize=address чи -fsanitize=thread
- -fsanitize=undefined – пошук випадків невизначеної поведінки.
- Відповідні опції слід передавати і компілятору і лінкеру!
- Див. також valgrind та санітайзери clang.
- Взагалі, санітайзери спершу з'явилися в clang, а віднедавна їх підтримує і MSVC: `https://devblogs.microsoft.com/cppblog/asan-for-windows-x64-and-debug-build-support/`.
- Есей: "Fun with -fsanitize=undefined and Picolibc".

GCC – драйвер компіляції

- Фактично, виконавчий файл gcc чи g++ – драйвер, який викликає інші виконавчі файли.
- Серія статей із оглядом його особливостей від Fabien Sanglard[5]:
  - `https://fabiensanglard.net/dc/index.php` – "Introduction".
  - `https://fabiensanglard.net/dc/driver.php` – "The Compiler Driver", стадії компіляції, від препроцесингу до лінкування.
  - `https://fabiensanglard.net/dc/cpp.php` – "The Preprocessor", зокрема про precompiled headers,
  - `https://fabiensanglard.net/dc/cc.php` – "The Compiler", зокрема структура: frontend-optimizer-backend.
  - `https://fabiensanglard.net/dc/ld.php` – "The Linker", зокрема згадує скрипти лінкера,
  - `https://fabiensanglard.net/dc/loader.php` – "The Loader", про те, як відбувається завантаження процесу.
- Деякі аспекти ми також зустрінемо в розмовах про асемблер.

[5] Зацитую автора: "*A contrasting experience was to learn how to use the tools to turn my programs into executable. It was a painfully slow and deeply unpleasant process where knowledge was gathered here and there after trial, errors, and a lot of time spent on search engines. It felt like acquiring the same level of comfort to use a compiler took significantly more than learning the language.*".

Робота із Core dump – Linux

- DRAFT
- https://opensource.com/article/20/8/linux-dump
- https://en.wikipedia.org/wiki/Core_dump
- https://www.jetbrains.com/help/clion/core-dump-debug.html – підтримка в CLion.
- https://interrupt.memfault.com/blog/linux-coredumps-part-1, https://interrupt.memfault.com/blog/linux-coredumps-part-2, https://interrupt.memfault.com/blog/linux-coredumps-part-3, https://interrupt.memfault.com/blog/gnu-build-id-for-firmware

GDB

- gdb, GNU DeBugger – дуже корисний інструмент. Хоча і з нюансами та елементами магії.
- Він консольний, але більшість GUI-дебагерів на сумісних системах використовують
- Розглянемо лише деякі, ключові (для нас) команди.
- "set disassembly-flavor intel" – за замовчуванням буде AT&T діалект.
- "set disassemble-next-line on" – зручніше виконувати команди по одній.
- Щоб не виконувати ці команди вручну, можна їх помістити в файл .gdbinit у поточній директорії. За замовчуванням, з міркувань безпеки, він ігноруватиме цей файл, щоб дозволити, слід додати в `/home/<username>/.gdbinit` щось типу "set auto-load safe-path /" (можна вказати більш обмежений безпечний шлях).
- Передати опції командного рядка програмі, яка зневаджується: `gdb --args executablename arg1 arg2 arg3`.

- Команди дозволяється скорочувати, поки вони залишаються унікальними.
- Наприклад, break – задати точку зупинки. Однак, можна писати і просто b.
- Аргумент break – адреса або мітка: "break *0x400", "b label".
- Для наших цілей – навчальних експериментів, під Linux, зручною є наступна комбінація точок зупинки:

```
b _start
b __libc_start_main
b main
```

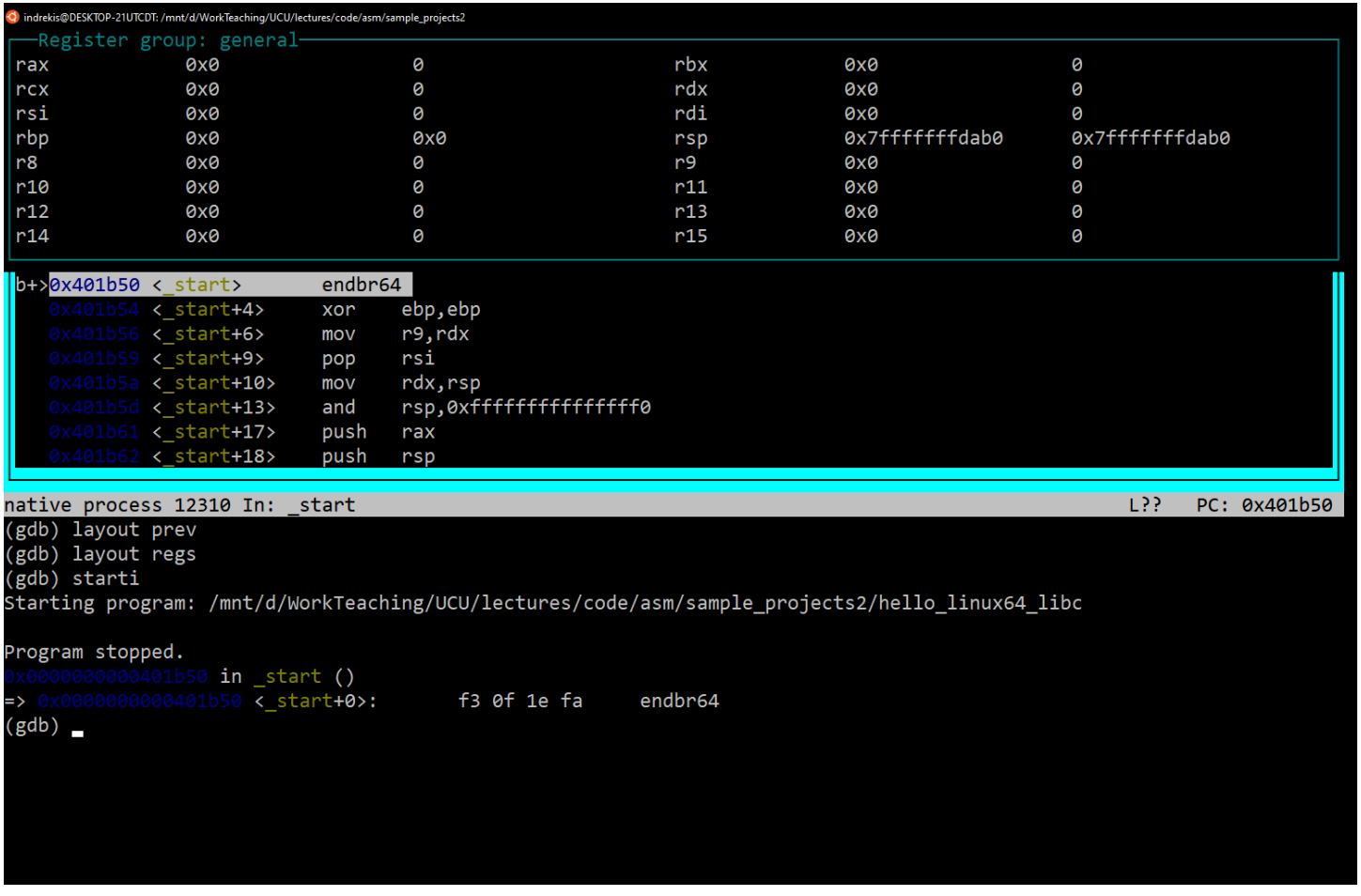
- run чи r – запустити програму. Вона виконуватиметься до першої точки зупинки або до завершення.
- nexti – виконати наступну команду. Якщо це виклик підпрограми – не заходити в неї.
- stepi – виконати наступну команду, заходячи в підпрограму за потреби.
- cont – продовжити виконання до наступної точки зупинки.
- return – вийти з підпрограми.

- starti (8.0 gdb+) – йти до першої інструкції програми. (Без того чисто асемблерну програму може бути важко запустити).
- "i r" – вивести вміст регістрів.
- "i r esp", "i r esp ebp rip"– вивести вміст конкретного регістра/регістрів.
- disas – дизасемблювати поточну функцію.
- Якщо немає нормального кадру стеку, disas може відмовитися працювати. Допомагає такий підхід: `disas 0x04000b0, +64` – дизасемблювати наступних 64 байти.
- "x" (eXplore) – вивести вміст пам'яті. Містить специфікатори формату. Наприклад, вивести C-стрічку: `x /s 0x4010d1`, `x /xw 0x4010d1` – вивести слово (w) у шістнадцятковому коді (x). Детальніше див. вивід "help x" (те ж стосується й інших команд).
- "info proc mappings" – карта пам'яті процесу.
- Офіційна документація: `https://sourceware.org/gdb/onlinedocs/gdb/index.html`.

Посилання

- `https://interrupt.memfault.com/blog/automate-debugging-with-gdb-python-api` — дуже потужний інструмент!

GDB TUI



- Документація: `https://sourceware.org/gdb/onlinedocs/gdb/TUI-Commands.html#TUI-Commands`.

GUI Debuggers

- x64dbg – GUI-дебагер (як 64-бітовий, так і 32-бітовий). `https://x64dbg.com` – для Windows.
- `https://github.com/eteran/edb-debugger` – кросплатформовий (Linux, FreeBSD, OpenBSD, OSX, Windows), але може довестися компілювати самостійно.
- Cutter – інтерактивний дизасемблер, дебагер і т.д.: `https://cutter.re`.
- Ще один дуже потужний інструмент – Radare2, `https://rada.re/r/`.
- `https://www.tecmint.com/best-hex-editors-for-linux/`, `https://mh-nexus.de/en/hxd/`
- `https://www.jetbrains.com/help/clion/core-dump-debug.html` – CLion підтримує для відладки GDB, а також підтримує core-dump “посмертне” відлагодження.

FASM/FASMG

- Директиви FASM: `https://flatassembler.net/docs.php?article=manual#2.2`. TODO: структури, макроінструкції, локальні мітки, ".if" (`https://flatassembler.net/docs.php?article=win32#2.2`) тощо.
- Під Windows та під Linux, якщо ви встановлювали FASM вручну, зручно додати шлях до нього в PATH. Зробити це в bash можна так:

```
export PATH=<path-to-fasm-dir>/fasmw172:$PATH
```

- Пам'ятайте, що різні емулятори POSIX систем під Windows використовують трішки різні способи представити шлях. Наприклад, `D:\MyDir` виглядатиме:

```
/d/MyDir -- під MSYS
/mnt/d/MyDir -- під Cygwin
/mnt/d/MyDir -- під WSL
```

- А під cmd взагалі слід писати:

```
set PATH=[disk]:\<path-to-fasm-dir>\fasmw172:%PATH%
```

- Також, особливо під Windows, із його розлогими дефініціями API, варто створити таку змінну:

```
export INCLUDE=<path-to-fasm-dir>/fasmw172/INCLUDE
```

- WSL має один нюанс – ця підсистема (поки?) не підтримує 32-бітових програм. Однак, менеджер пакетів інсталює fasm завжди як 32-бітовий бінарник! Тільки такі є і на офіційному сайті. Тому, для використання під WSL, слід скомпілювати його самостійно, (на іншій системі – де Ви зможете запустити fasm тим чи іншим способом). Джерельні тексти є у відповідних архівах, вам знадобляться ці: `<fasm_dir>/source/Linux/x64`.
- WSL2 підтримує, хоча може потребувати інсталяції відповідних пакетів, однак, скомпілювати 64-бітову версію виглядає раціональнішим.
- Там же містяться ще цікаві інструменти. Поміж них я б виділив listing (`<fasm_dir>/tools/win32` чи `<fasm_dir>/tools/libc`). Скомпілюйте цю утиліту та помістіть туди, звідки вам буде зручно її викликати. Отримати лістинг вашої програми можна буде так:

```
fasm ./libcdemo.asm -s ./libcdemo.dbg && \
  <path-to-l>/listing ./libcdemo.dbg ./libcdemo.lst
```

- Oftopics: `https://board.flatassembler.net/topic.php?t=10609` – Інтерпретатор befungi на FASM.

Інші асемблери

- NASM
- YASM
- І т.д.
- Цікавий список: `https://github.com/jaspergould/awesome-asm`.

pahole

- Компілятору заборонено переставляти елементи структур, а переставляння елементів класів дуже обмежене[6], однак, через потребу у вирівнюванні, це може спричиняти неефективне використання пам'яті.
- Утиліта pahole дозволяє діагнозувати та вирішувати такі проблеми, оптимізуючи розташування в пам'яті елементів структур: "pahole: Analysing Memory Layout of Complex Data Structures With Ease".

[6] В загальному випадку, компілятор C++, до https://quuxplusone.github.io/blog/2022/03/04/non-standard-layout-guarantees/ міг переставляти між собою public, protected і private блоки.

- "FIYA – Flamegraphs in Your App" – бібліотека, майже header-only, для створення flamegraph безпосередньо з коду. Корисне, зокрема, в embedded.

- sysdig – "Linux system exploration and troubleshooting tool with first class support for containers".

valgrind

- "A brief and incomplete comparison of memory corruption detection tools".
- "Memory corruption from outside the process looks like space aliens" – есей на тему.

BugChecker

- "BugChecker" – kernel-level дебагер, що дозволяє відлагоджувати код в ядрі на тому ж комп'ютері, емулюючи відлагодження по UART[7].

[7] Сучасний аналог славнозвісного SoftICE.

